C#-Basics

Objective

- Difference between Java and C#
- Console Applications
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Comments
- Parsing
- Input/output
- First C# Program
- Conditional Statements
- Iterative Statements
- arrays
- cmd line Parameter

Difference between Java and C#

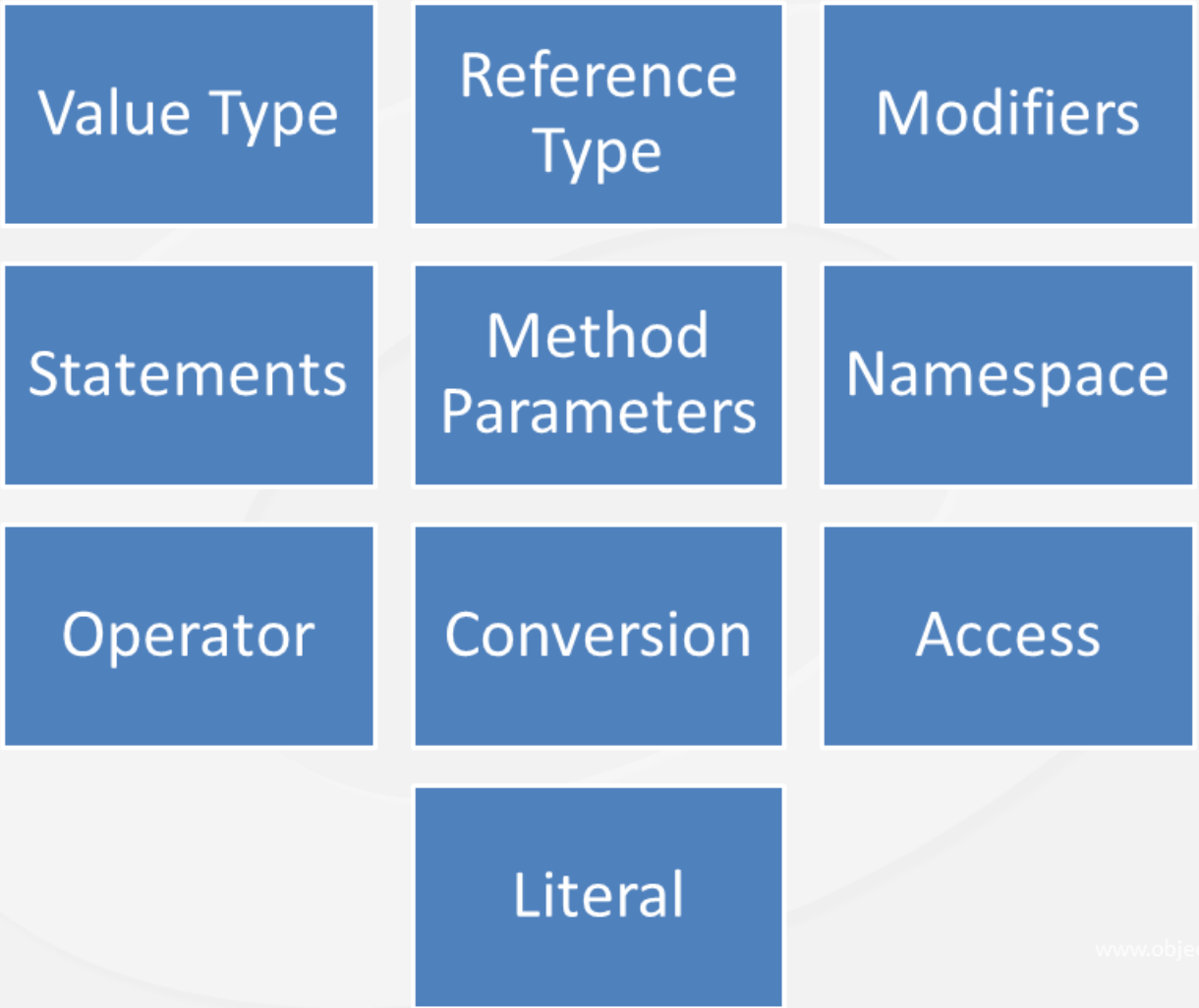
OBJECT TECHNOLOGIES

Operator Overloading	C# supports operator overloading for multiple operators.	Java does not support operator overloading.
Runtime Environment	C# supports CLR(Common Language Runtime).	Java supports JVM(Java Virtual Machine).
API Control	C# API are controlled by open source community.	Java API are also controlled by open community process.
Public Classes	In C#, there can be many public classes inside a source code.	In Java there can be only one public class inside a source code otherwise there will be compilation error.
Checked Exceptions	C# does not supports for checked exception. In some cases checked exceptions are very useful for smooth execution of program.	Java supports both checked and unchecked exceptions.
Platform Dependency	C# is cross-platform and runs on both Windows & Unix based systems.	Java is a robust and platform independent language. Platform independency of Java is through JVM.
Pointers	In C# pointers can be used only in unsafe mode.	Java does not supports anyway use of pointers.
Conditional Compilation	C# supports for conditional compilation.	Java does not supports for conditional compilation.
goto statement	C# supports for goto statement.	Java does not supports for goto statement. Use of goto statement will cause error in Java code.
Structure and Union	C# supports structures and unions.	Java doesn't support structures and unions.
Floating Point	C# does not supports strictfp keyword that means it result of floating point numbers may not be guaranteed to be same across all platforms.	Java supports strictfp keyword that means its result for floating point numbers will be same for various platform.

Keywords

- Keywords or Reserved words are the words in a language that are used for some internal process or represent some predefined actions. These words are therefore not allowed to use as variable names or objects.
- There are total **78** keywords in C#.
- Keywords in C# is mainly divided into 10 categories.

Types of Keywords



Contextual Keywords

- These are used to give a specific meaning in the program. Whenever a new keyword comes in C#, it is added to the contextual keywords, not in the keyword category. This helps to avoid the crashing of programs which are written in earlier versions.

  Important Points:

- These are not reserved words.
- It can be used as identifiers outside the context that's why it named contextual keywords.
- These can have different meanings in two or more contexts.
- There are total 30 contextual keywords in C#.

Contextual Keywords Example

add	get	partial(method)
alias	global	remove
ascending	group	select
async	into	set
await	join	value
by	let	var
descending	nameof	when
dynamic	on	where
equals	orderby	where
from	partial(type)	yield

Identifiers

- Identifiers are the name given to entities such as variables, methods, classes, etc. They are tokens in a program which uniquely identify an element.

Rules for Naming an Identifier

1. An identifier can not be a C# keyword.
2. An identifier must begin with a letter, an underscore or @ symbol. The remaining part of identifier can contain letters, digits and underscore symbol.
3. Whitespaces are not allowed. Neither it can have symbols other than letter, digits and underscore.
4. Identifiers are case-sensitive. So, getName, GetName and getname represents 3 different identifiers.

Examples

Identifiers	Remarks
number	Valid
calculateMarks	Valid
hello$	Invalid (Contains $)
name1	Valid
@if	Valid (Keyword with prefix @)
if	Invalid (C# Keyword)
My name	Invalid (Contains whitespace)
_hello_hi	Valid

Variables

- A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.
- It is a way to represent memory location through symbol so that it can be easily identified.

  Syntax:

  type variable_name = value;

  Or

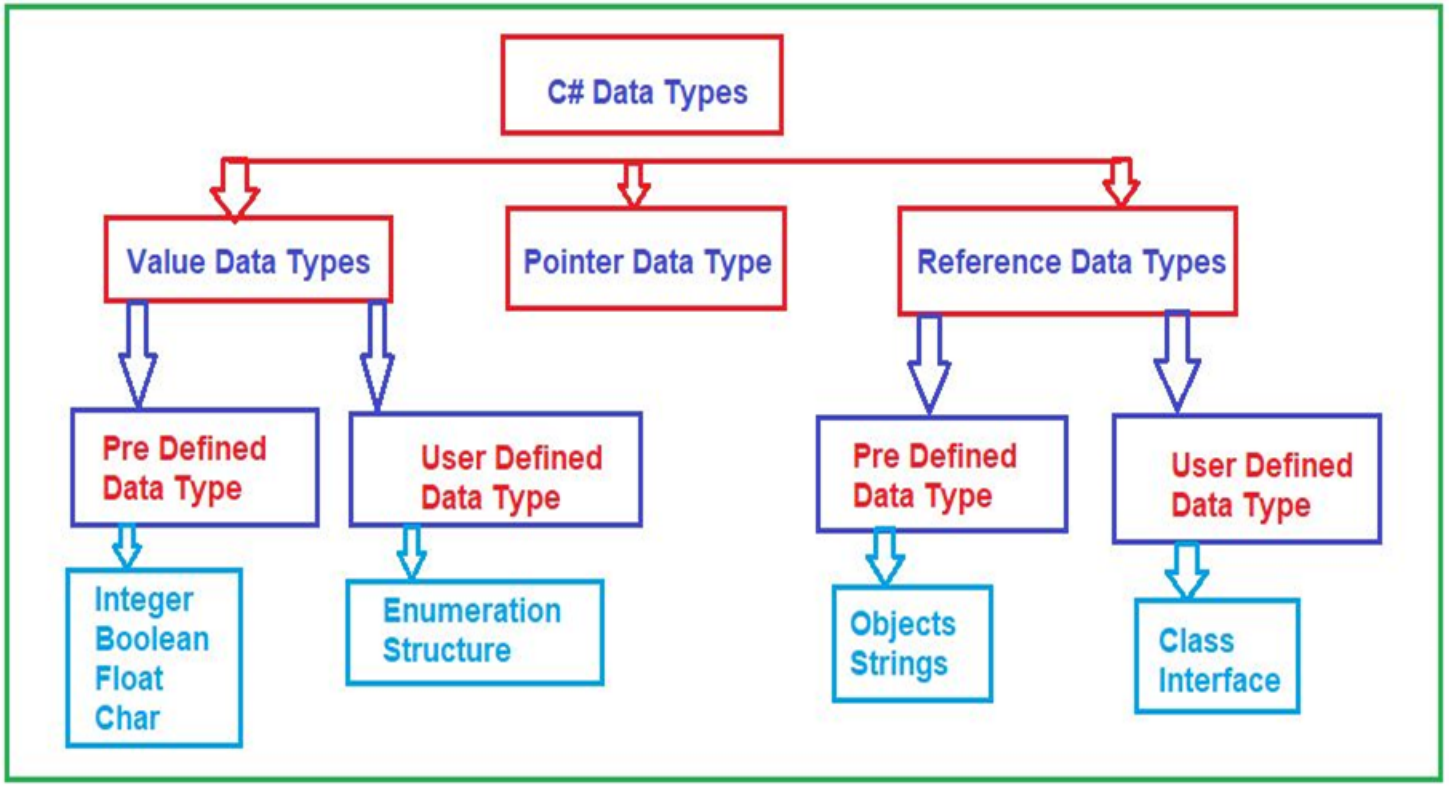
  type variable_names;

  Example:

```
char ch = 'h'; // Declaring and Initializing character variable
int a, b, c; // Declaring variables a, b and c of int type
```

Var	Dynamic
It is introduced in C# 3.0.	It is introduced in C# 4.0
The variables are declared using var keyword are statically typed.	The variables are declared using dynamic keyword are dynamically typed.
The type of the variable is decided by the compiler at compile time.	The type of the variable is decided by the compiler at run time.
The variable of this type should be initialized at the time of declaration. So that the compiler will decide the type of the variable according to the value it initialized.	The variable of this type need not be initialized at the time of declaration. Because the compiler does not know the type of the variable at compile time.
If the variable does not initialized it throw an error.	If the variable does not initialized it will not throw an error.
It support intelliSense in visual studio.	It does not support intelliSense in visual studio
var myvalue = 10; // statement 1	dynamic myvalue = 10; // statement 1
myvalue = “Welcome”; // statement 2	myvalue = “Welcome”; // statement 2
Here the compiler will throw an error because the compiler has already decided the type of the myvalue variable using statement 1 that is an integer type. When you try to assign a string to myvalue variable, then the compiler will give an error because it violating safety rule type.	Here, the compiler will not throw an error though the type of the myvalue is an integer. When you assign a string to myvalue it recreates the type of the myvalue and accepts string without any error.
It cannot be used for properties or returning values from the function. It can only used as a local variable in function.	It can be used for properties or returning values from the function.

Data Types


C# Data Types
Value Data Types
Pointer Data Type
Reference Data Types
Pre Defined Data Type
User Defined Data Type
Pre Defined Data Type
User Defined Data Type
Integer
Boolean
Float
Char
Enumeration
Structure
Objects
Strings
Class
Interface

Default Values

- Every data type has a default value. Numeric type is 0, boolean has false, and char has '' as default value.
- Use the default(typename) to assign a default value of the data type or C# 7.1 onward, use default literal.

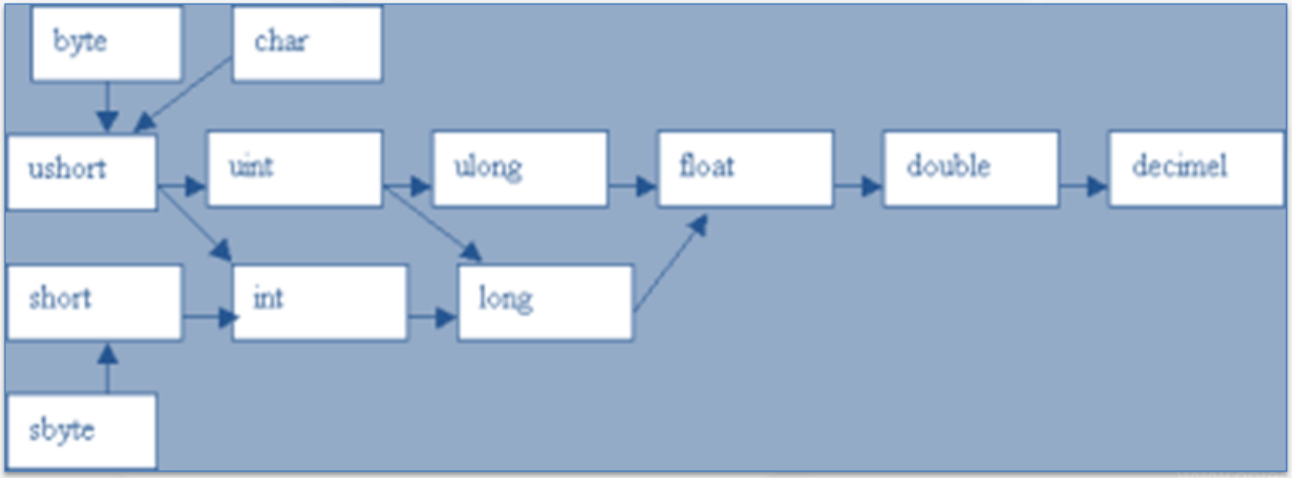
```
int i = default(int); // 0
float f = default(float);// 0
decimal d = default(decimal);// 0
bool b = default(bool);// false
char c = default(char);// ''
```

```
// C# 7.1 onwards
int i = default; // 0
float f = default;// 0
decimal d = default;// 0
bool b = default;// false
char c = default;// ''
```

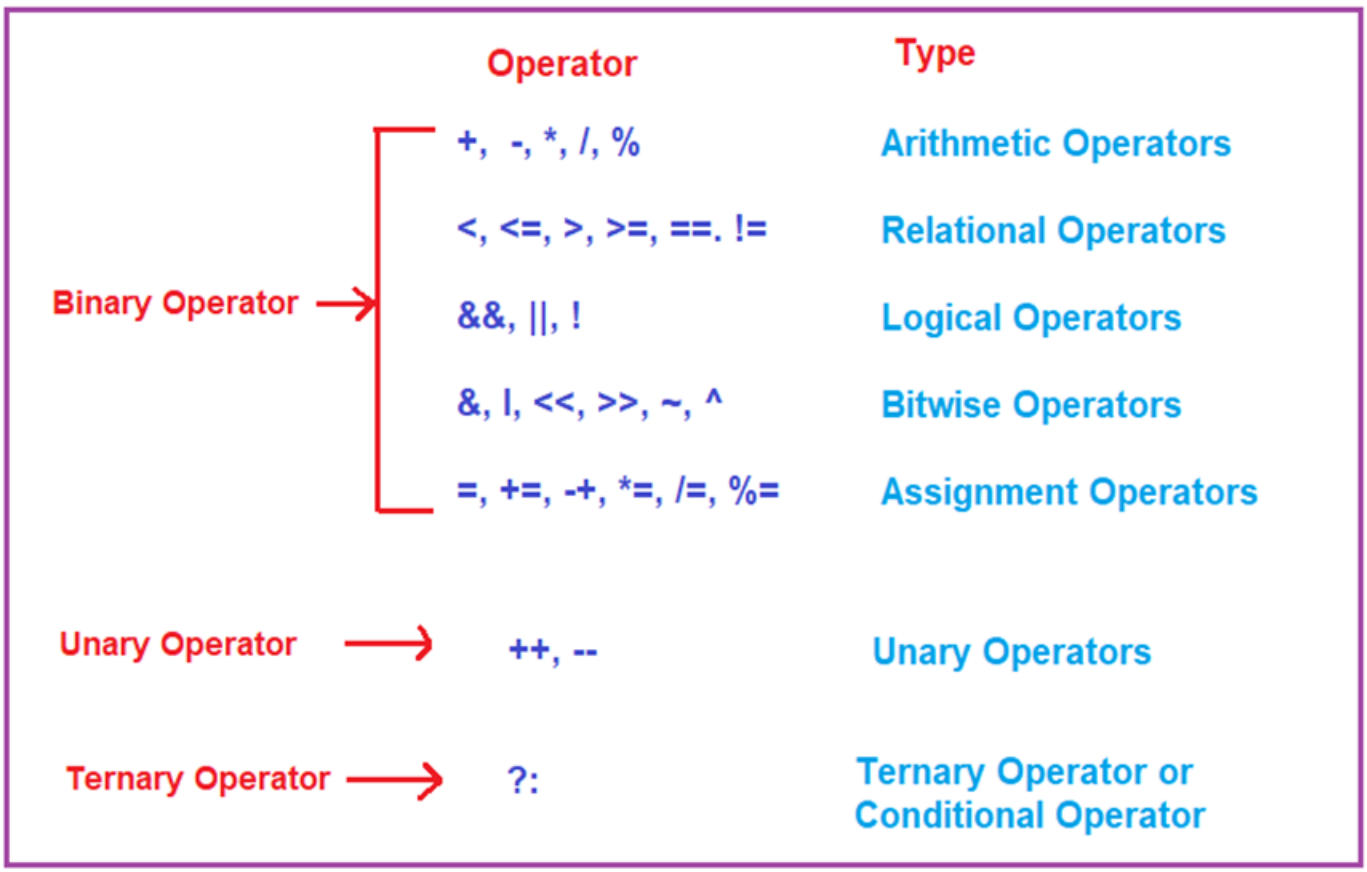
Conversions

- The values of certain data types are automatically converted to different data types in C#. This is called an implicit conversion. Example:

```
//Implicit Conversion
int i = 345;
float f = i;
Console.WriteLine(f); //output: 345
```



Operators



Comments

```
class Program
{
    0 references
    static void Main(String[] args)
    {
        // THIS IS SINGLE LINE COMMENT

        /*
            string name = Console.ReadLine();
            Console.WriteLine("Welcome {0}", name);
            Console.Read();
        */

    }
}
```

Shortcut: You can use **Ctrl+K, Ctrl+C** and **Ctrl+K, Ctrl+U** to Comment or Uncomment selected lines of text.

Parsing

- Parse() converts the string data type to another data type.
- For example, a parsing operation is used to convert a string to a floating-point number or to a date-and-time value.
  Syntax
- To convert string into any data type, we use the following syntax:
  data_type.Parse(string);

Example:

```
string var_string = "4.5";
Console.WriteLine(int.Parse("4") + 3);
Console.WriteLine(float.Parse(var_string) + 4.4f)
```

The int.Parse() method converts the string, "4“ into an integer value 4.

The float.Parse() method converts the string "4.5" into a float value 4.5

TryParse() method

- TryParse is .NET C# method that allows you to try and parse a string into a specified type.
- It returns a boolean value indicating whether the conversion was successful or not.
- If conversion succeeded, the method will return true and the converted value will be assigned to the output parameter.
- If conversion failed, the return value will be false and the output parameter will not change value (default value).

```
if (int.TryParse(inputString, out int number))
{
    // call(number);
}
else
{
    Console.WriteLine("Provided inputString was invalid.");
}
```

- **Out parameter**
- The out parameter is how the TryParse method passes the parsed value back to you.
- It's a parameter of type T, where T is the type that you are trying to parse the string into.
- **Static method**
- TryParse is a static method, so it can be called without creating instance of the class.
- **TryParse is used for many types:**

int	decimal	bool	DateTime	char
Enum	string	byte	ushort	

Input/output

- **C# Output**

  System.Console.WriteLine() OR

  System.Console.Write()

- Here, System is a namespace, Console is a class within namespace System and WriteLine and Write are methods of class Console.

```
Console.WriteLine("Value = " + val);
Console.WriteLine("Value = {0}", val);
```

```
int firstNumber = 5, secondNumber = 10, result;
 result = firstNumber + secondNumber;
Console.WriteLine("{0} + {1} = {2}", firstNumber, secondNumber,
result);
```

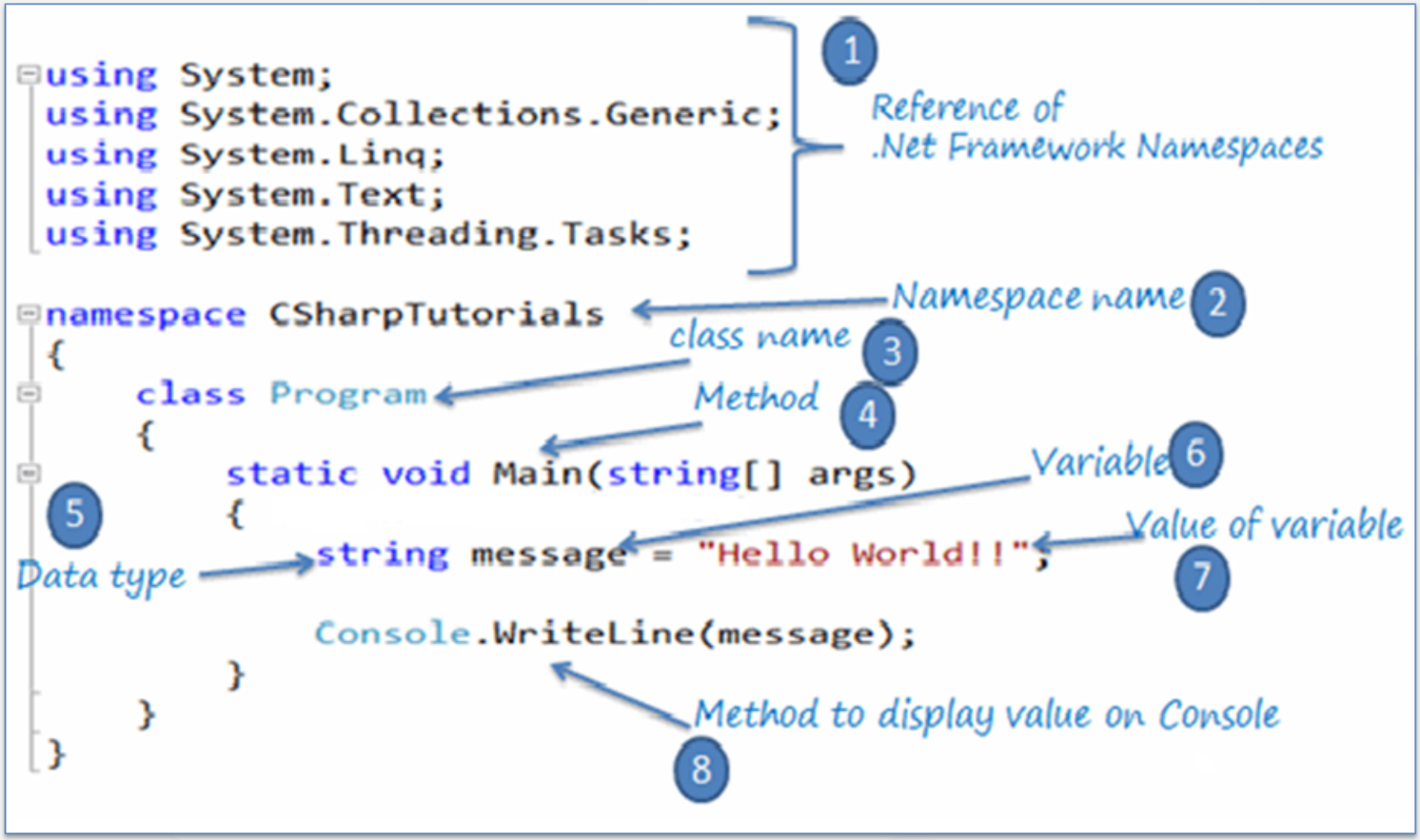
- **C# Input**
- Console.ReadLine(), Console. Read(), Console. ReadKey()

- **Difference between ReadLine(), Read() and ReadKey() method:**
- ReadLine(): The ReadLine() method reads the next line of input from the standard input stream. It returns the same string.

- Read(): The Read() method reads the next character from the standard input stream. It returns the ascii value of the character.

- ReadKey(): The ReadKey() method obtains the next key pressed by user. This method is usually used to hold the screen until user press a key.

First C# Program



Conditional Statements

- if
- if-else
- if-else-if
- Nested if
- Switch
- Nested switch
- Ternary/Conditional operator

if Statement

```
if (condition)
    statement;
```

if...else Statement

```
if (condition)
    statement;
else (condition)
    statement;
```

if...else if...else Statement

```
if (condition)
    statement:
else if (condition)
    statement;
else
    statement;
```

if...else if...else Statement

```
if (condition)
    statement;
else if (condition)
    statement;
else if (condition)
    statement;
else
    statement;
```

Switch

```
switch(variable)
{
    case 1:
            //execute your code
    break;
    case n:
            //execute your code
    break;
    default:
            //execute your code
    break;
}
```

Switch case

- Important points to remember about Switch case:
- In C#, duplicate case values are not allowed.
- The data type of the variable in the switch and value of a case must be of the same type.
- The value of a case must be a constant or a literal. Variables are not allowed.
- The break in switch statement is used to terminate the current sequence.
- The default statement is optional and it can be used anywhere inside the switch statement.
- Multiple default statements are not allowed.

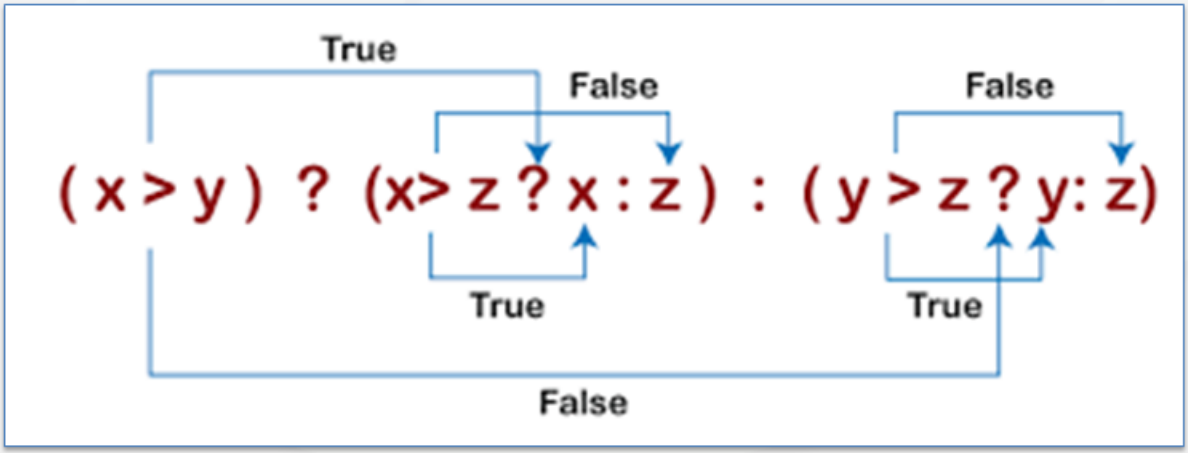
Ternary Operator

The value to be returned if the condition returns true.

condition ? value if true : value if false

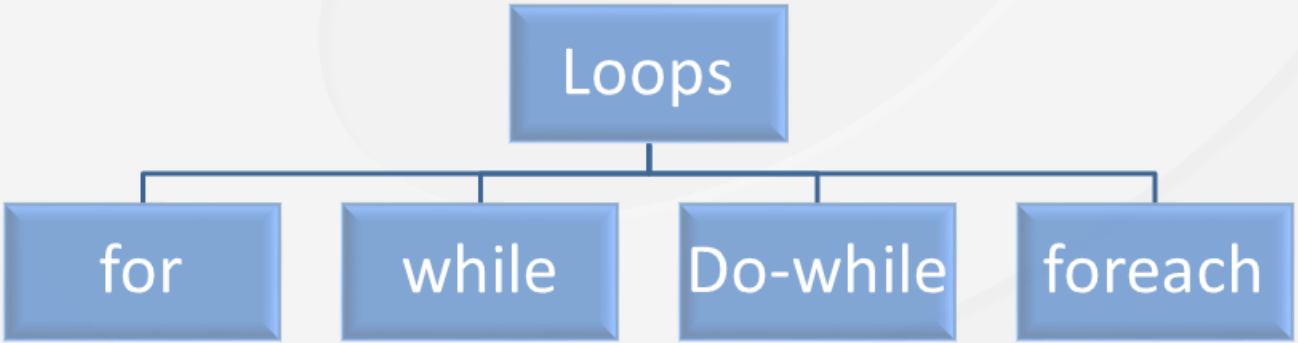
The condition to be tested. Should return either true or false.

The value to be returned if the condition returns false.

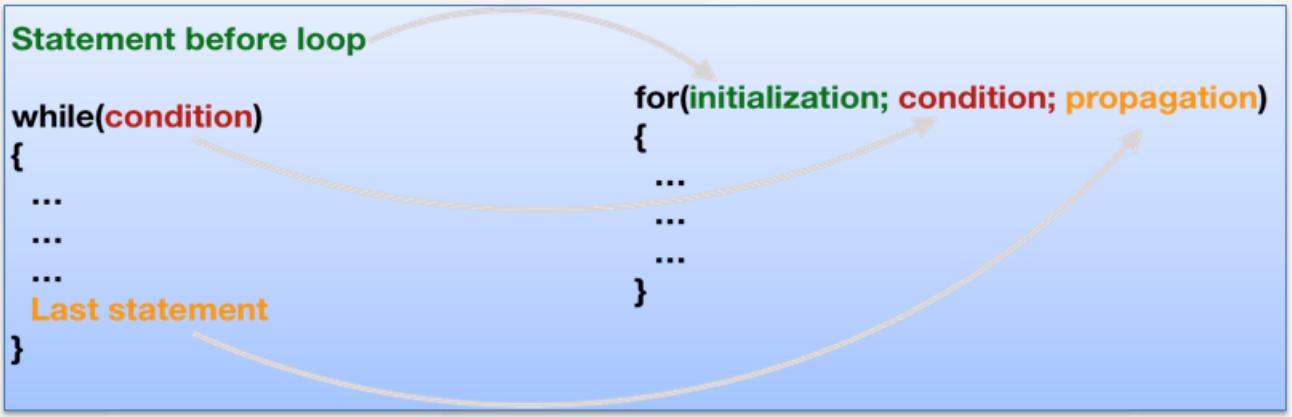


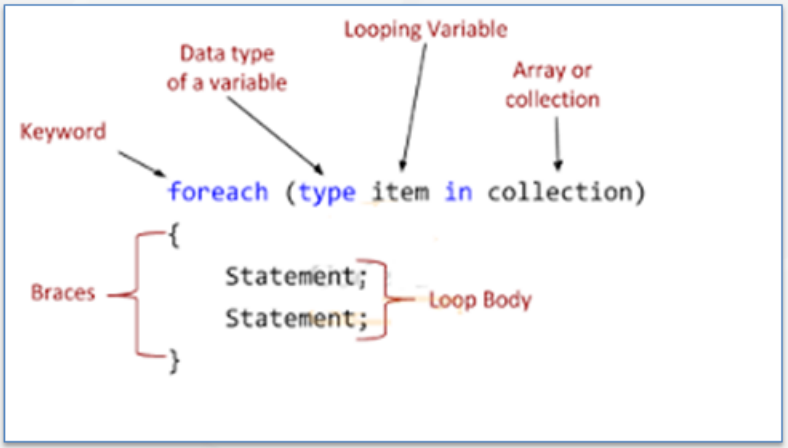
Iterative Statements

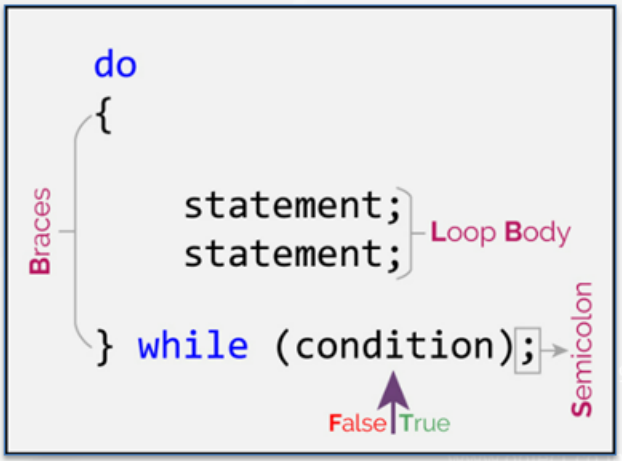
- C# iteration statement (loop) is a block of code that will repeat itself over and over again until a given condition is true or some terminating condition is reached.
- The condition is anything that you want the program to evaluate before each and every time it loops.
- Whenever the condition is true, the program knows to repeat. Once that condition is false, then it's time to stop repeating.



Syntax


Statement before loop
while(condition)
{
...
...
...
Last statement
}
for(initialization; condition; propagation)
{
...
...
...
}

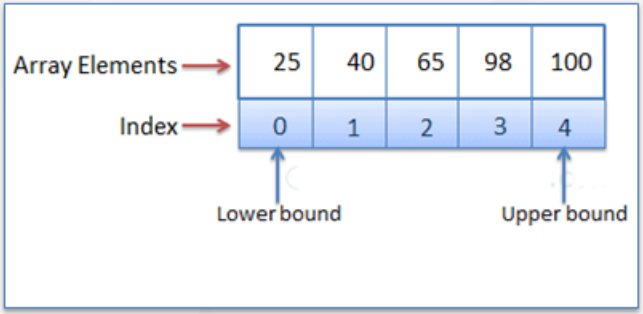

Looping Variable
Data type of a variable
Array or collection
Keyword
foreach (type item in collection)
{
Statement;
Statement;
}
Braces
Loop Body


do
{
statement;
statement;
} while (condition);
Braces
Loop Body
Semicolon
False
True

Arrays

- **What is Array?**

1. Array are the homogeneous(similar) collection of data types.
2. Array Size remains same once created.
3. Simple Declaration format for creating an array
   type [ ] identifier = new type [integral value];



1. **Advantages of Arrays:**

- Random access to values stored at different memory locations.
- Easy data manipulation like Data sorting, Data traversing or other operations.
- Optimization of code.

Types Of Arrays In C#

- 1 dimensional or Single Dimensional Array
- Multi-Dimensional Array
- Jagged Array

- **Array Declaration**:

```
int[ ] integerArray;
string[ ] stringArray;
bool[ ] booleanArray;
string[ ] student = new string[ 3 ];
```

- **Array Initialization**:

```
string[ ] student = new string[ 3 ]{“stud1”, “stud2”, “stud3”};
string[ ] student = {“student1”, “student2”, “student3”};
```

Accessing Array Elements

- Access Array Elements using Indexes

```
int[] evenNums = new int[5];
evenNums[0] = 2;
evenNums[1] = 4;
Console.WriteLine(evenNums[0]); //prints 2
Console.WriteLine(evenNums[1]); //prints 4
```

- **Accessing Array using for Loop**
- Use the length property of an array in conditional expression of the for loop.

```
int[] evenNums = { 2, 4, 6, 8, 10 };
for(int i = 0; i < evenNums.Length; i++)
        Console.WriteLine(evenNums[i]);
```

- **Accessing Array using foreach Loop**

```
int[] evenNums = { 2, 4, 6, 8, 10};
string[] cities = { "Mumbai", "London", "New York" };
foreach(var item in evenNums)
    Console.WriteLine(item);
foreach(var city in cities)
    Console.WriteLine(city);
```

MOST COMMON PROPERTIES OF ARRAY CLASS		
Properties	Explanation	Example
Length	Returns the length of array. Returns integer value.	int i = arr1.Length;
Rank	Returns the rank of the Array.Rank is the number of dimensions of an array.	int i = arr1.Rank; 1-D array returns 1 2-D array returns 2
IsFixedSize	Check whether array is fixed size or not. Returns Boolean value	bool i = arr.IsFixedSize;
IsReadOnly	Check whether array is ReadOnly or not. Returns Boolean value	bool k = arr1.IsReadOnly;

MOST COMMON FUNCTIONS OF ARRAY CLASS		
Function	**Explanation**	**Example**
Sort	Sort an array	Array.Sort(arr);
Clear	Clear an array by removing all the items	Array.Clear(arr, 0, 3);
GetLength	Returns the number of elements	arr.GetLength(0);
GetValue	Returns the value of specified items	arr.GetValue(2);
IndexOf	Returns the index position of value	Array.IndexOf(arr,45);
Copy	Copy array elements to another elements	Array.Copy(arr1,arr1,3);

Multi dimensional array

```
public static void Main(String[] args)
{
    // Mulidimentional Array Declaration
    int[,] arr2d_1; // two-dimensional array
    int[,,] arr3d; // three-dimensional array
    int[,,,] arr4d; // four-dimensional array
    int[,,,,] arr5d; // five-dimensional array

    //Array Initialisation
    int[,] arr2d = new int[3, 2]{ {1, 2},{3, 4},{5, 6} };

    // or
    int[,] arr2d_2 = { {1, 2}, {3, 4},{5, 6} };

    //Accessing values
    Console.WriteLine(arr2d[0, 0]); //returns 1
    Console.WriteLine(arr2d[0, 1]); //returns 2
    Console.WriteLine(arr2d[1, 0]); //returns 3
    Console.WriteLine(arr2d[1, 1]); //returns 4
}
```

Cmd line parameter

1. Create Firstprog.cs file in notepad with following code.

```
using System;
public class Myclass
{
    public static void Main(String[] args)
    {
        Console.Write("Welcome");
        if(args.Length >0)
        {
          foreach(var p in args)
              Console.WriteLine(p);
        }
    }
}
```

2. Open .net cmd prompt and to compile code use csc command and filename.cs.

C:\Snehal\Demo>**csc FirstProg.cs**

C:\Snehal\Demo>FirstProg.exe

C:\Snehal\Demo>FirstProg.exe Hi Snehal Welcome to know It!!!

```
C:\Snehal\Demo>csc FirstProg.cs
Microsoft (R) Visual C# Compiler version 4.4.0-6.22559.4 (d7e8a398)
Copyright (C) Microsoft Corporation. All rights reserved.


C:\Snehal\Demo>FirstProg.exe
Welcome
C:\Snehal\Demo>FirstProg.exe Hi Snehal Welcome to know It!!!!
WelcomeHi
Snehal
Welcome
to
know
It!!!!
```

code compilation

Executing of Code

Passing Cmd line argument

Summary

- Difference between Java and C#
- Console Applications
- Keywords
- Identifier
- Variables
- Data Types
- Operators
- Comments
- Parsing
- Input/output
- First C# Program
- Conditional Statements
- Iterative Statements
- arrays
- cmd line Parameter

Thank You